

POSTGRESQL SORGU PERFORMANSI VE ÖNERİ MOTORU

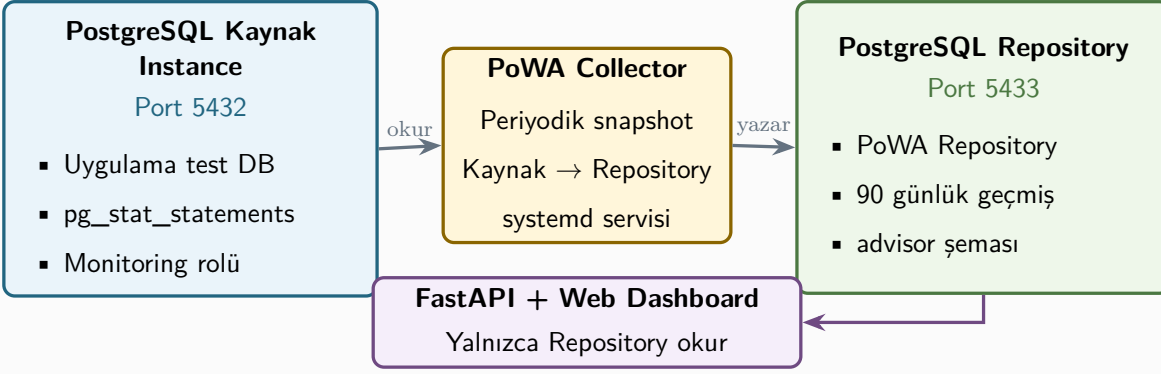
İlk İterasyon Teknik Mimari, Kurulum ve Uygulama Dokümantasyonu

Sürüm 1.1

22 Temmuz 2026

Belge amacı: kurulum, geliştirme, test ve ilerideki iterasyonlar için tek referans doküman oluşturmak.

TEK TEST SUNUCUSU



Belge Kontrolü

Alan	Değer
Belge adı	PostgreSQL Sorgu Performansı ve Öneri Motoru - İlk İterasyon
Sürüm	1.1
Durum	İlk iterasyon onaylı teknik tasarım ve uçtan uca kurulum runbook'u
Kapsam	Tek test sunucusu üzerinde iki PostgreSQL instance, PoWA Collector, Repository, FastAPI ve dashboard
Kapsam dışı	Otomatik CREATE/DROP INDEX, SQL rewrite, gereksiz JOIN kaldırma, canlıya otomatik müdahale
Hedef kullanıcılar	Yazılım ekibi, DBA, sistem yöneticisi, test ekibi, teknik karar vericiler

Karar özeti: İlk iterasyonun temel çıktısı “hangi sorgulara önce bakılmalı?” sorusudur. “Nasıl düzeltilmeli?” önerileri ikinci iterasyona bırakılmıştır.

İçindekiler

Belge Kontrolü	1
1. Yönetici Özeti	5
2. Amaç, Kapsam ve Başarı Tanımı	5
2.1 Amaç	5
2.2 İlk iterasyon kapsamı	5
2.3 İlk iterasyon kapsamı dışında kalanlar	6
3. Mimari Kararlar	6
4. Genel Sistem Mimarisi	7
4.1 Kaynak instance - Port 5432	7
4.2 Repository instance - Port 5433	7
4.3 PoWA Collector	7
5. Bileşenler ve Sorumluluklar	8
6. Kurulum Öncesi Gereksinimler	8
6.1 Donanım ve işletim sistemi	8
6.2 Ortam keşif komutları	9
6.3 Kurulum parametreleri	9
6.4 Resmî indirme adresleri ve sürüm sabitleme	9
6.5 Tercih edilen kurulum rotası	10
7. Baştan Sona Paketli Kurulum Runbook'u	11
7.1 Kurulumdan önce yedek ve keşif	11
7.2 Debian / Ubuntu üzerinde PGDG deposunu hazırlama	11
7.3 Debian / Ubuntu paketlerinin kurulması	12
7.4 Repository cluster'ını 5433 portunda oluşturma	12
7.5 Kaynak instance'ta pg_stat_statements etkinleştirme	12
7.6 Kaynak instance üzerinde dedicated PoWA veritabanı	13
7.7 Collector PostgreSQL rolünü oluşturma	13
7.8 Repository veritabanı ve PoWA Repository kurulumu	14
7.9 pg_hba.conf ve localhost erişimi	14
7.10 Repository'ye kaynak sunucuyu kaydetme	15
7.11 PoWA Collector'ı virtual environment ile kurma	15
7.12 Collector systemd servisi	16

7.13 Collector ve repository doğrulaması	17
7.14 RHEL / Rocky / AlmaLinux kurulum farkları	17
8. Manuel ve Offline Kurulum	18
8.1 Offline paket setini internet erişimli makinede hazırlama	18
8.2 Offline Collector kurulumu	19
8.3 PoWA Archivist'i kaynak koddan derleme	19
8.4 Checksum ve kurulum kanıtı	20
8.5 Kurulum sonrası geri alma	20
9. Güvenlik ve Yetkilendirme	20
10. Veri Modeli ve Repository Okuma Katmanı	21
10.1 PoWA tablolarını değiştirmeme prensibi	21
10.2 Analitik view katmanı	21
10.3 Repository okuma prensipleri	22
11. Backend Tasarımı (FastAPI)	22
11.1 Önerilen teknoloji seti	22
11.2 Proje yapısı	22
11.3 Temel endpointler	23
11.4 Örnek FastAPI ayarları	23
12. Web Arayüzü ve Dashboardlar	23
12.1 Genel bakış dashboardu	23
12.2 Sorgu listesi	24
12.3 Sorgu detay ekranı	24
12.4 Sistem sağlığı ekranı	24
13. Impact Score ve Regresyon Analizi	25
13.1 Puanın amacı	25
13.2 Regresyon hesabı	25
14. Sistem Sağlığı Kuralları	25
15. İşletim, İzleme, Yedekleme ve Retention	26
15.1 Servisler	26
15.2 Yedekleme	26
15.3 Retention ve kapasite	26
16. Test Planı ve Kabul Kriterleri	27

16.1 Kurulum testleri	27
16.2 Fonksiyonel kabul kriterleri	27
16.3 Performans testi	27
17. Hata Giderme ve Geri Alma	28
17.1 Geri alma planı	28
18. İkinci İterasyon Yol Haritası	28
19. Repo ve Kaynak Linkleri	29
Ek A. Kurulum Kontrol Listesi	30
Ek B. Örnek API Sözleşmesi	30
Ek C. Örnek Ekran Alanları	31

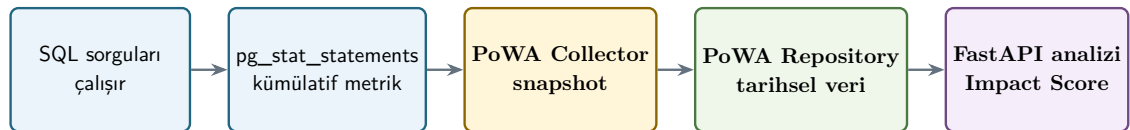
1. Yönetici Özeti

Bu doküman, PostgreSQL sorgu yükünü geçmişe dönük analiz eden ve yüksek etkili sorguları önceliklendiren ilk iterasyonun uçtan uca teknik planını tanımlar. Sistem tek bir fiziksel test sunucusunda çalışacak, ancak kaynak iş yükü ile geçmiş performans verisini mantıksal olarak ayırmak için iki PostgreSQL instance kullanılacaktır.

Kaynak instance (5432) uygulama test veritabanını barındırır ve `pg_stat_statements` ile normalize sorgu istatistiklerini üretir. Repository instance (5433) PoWA geçmişini saklar. PoWA Collector periyodik snapshot işlemini gerçekleştirir. Kendi FastAPI backendimiz yalnızca repository instance üzerinden okuma yapar; PoWA Web kullanılmaz.

- Tek fiziksel test sunucusu, iki ayrı PostgreSQL instance/cluster.
- Kaynak instance üzerinde minimum gerekli extension: `pg_stat_statements` ve PoWA remote snapshot fonksiyonları.
- PoWA Collector hazır bileşen olarak kullanılır; ilk iterasyonda özel collector yazılmaz.
- Repository geçmişi için başlangıç retention hedefi 90 gündür.
- Backend için önerilen teknoloji: Python 3.12+ ve FastAPI.
- Frontend teknoloji seçimi kritik değildir; React + TypeScript örnek referans olarak kullanılır.
- Kullanıcıya normalize SQL, yük metrikleri, dönem karşılaştırmaları, gerileme uyarıları ve sağlık göstergeleri sunulur.
- Hiçbir SQL önerisi otomatik olarak çalıştırılmaz; ilk iterasyonda CREATE/DROP INDEX veya query rewrite üretilmez.

İLK İTERASYON VERİ AKIŞI



Çıktı: en yüksek etkili sorgular, dönem karşılaştırmaları, gerileme uyarıları ve tablo sağlık göstergeleri

2. Amaç, Kapsam ve Başarı Tanımı

2.1 Amaç

PostgreSQL üzerinde çalışan normalize sorguları düzenli biçimde toplamak, tarihsel olarak saklamak, sorguları sistem etkisine göre sıralamak ve kullanıcıya hangi sorguların öncelikle incelenmesi gerektiğini açıklamak.

2.2 İlk iterasyon kapsamı

- `pg_stat_statements` üzerinden sorgu istatistiği toplama.
- PoWA Repository içinde tarihsel saklama.

- PoWA Collector ile periyodik snapshot.
- Son 1 saat, 24 saat, 7 gün ve 30 gün raporları.
- Normalize SQL metnini gösterme.
- Toplam süre, çağrı sayısı, ortalama süre, blok okuma, cache, temp ve WAL metrikleri.
- Impact Score ile sorgu önceliklendirme.
- Önceki eş dönemle performans regresyonu tespiti.
- Seq Scan yoğunluğu, dead tuple, autovacuum, uzun transaction ve lock bekleme göstergeleri.
- Kullanıcı notu, inceleme durumu ve audit kayıtları.

2.3 İlk iterasyon kapsamı dışında kalanlar

- CREATE INDEX / DROP INDEX önerisi üretme.
- Gereksiz JOIN, DISTINCT veya correlated subquery için otomatik rewrite.
- HypoPG ile sanal index testi.
- pg_qualstats ile predicate/WHERE/JOIN geçmişi.
- auto_explain ile otomatik plan toplama.
- Production üzerinde otomatik değişiklik uygulama.
- Tam otomatik DBA karar sistemi.

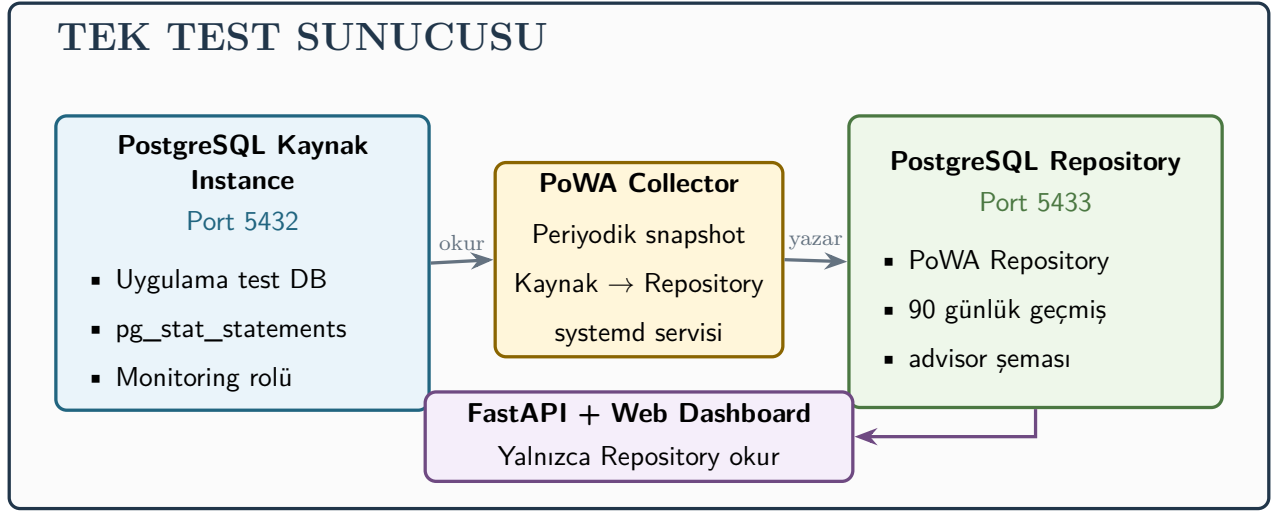
Başarı tanımı: Sistem, kullanıcıya en yüksek etkili sorguları doğru dönem ve bağlamla sıralayabiliyorsa ilk iterasyon değer üretir. Düzeltme SQL'i üretmemesi eksiklik değil, bilinçli kapsam kararıdır.

3. Mimari Kararlar

Karar alanı	Seçim	Gerekçe
Fiziksel yerleşim	Tek test sunucusu	Kurulumu sade tutmak ve pilotu hızlandırmak
Veritabanı yerleşimi	İki PostgreSQL instance: 5432 ve 5433	Repository yükünü uygulama DB'sinden ayırmak
Toplama modu	PoWA remote mode mantığı	PoWA Collector kullanmak ve background worker eklememek
Collector	Hazır PoWA Collector	Snapshot mekanizmasını tekrar yazmamak
Web arayüz	Kendi dashboardumuz	PoWA Web zorunlu değil; ürün deneyimi bize ait
Backend	Python + FastAPI	SQL ağırlıklı analiz için hızlı geliştirme
Kaynak DB erişimi	Backend doğrudan bağlanmaz	Analiz trafiğini kaynak DB'den ayırmak
Değişiklik uygulama	Otomatik yok	Riskli DDL/SQL işlemlerinde insan onayı

Karar alanı	Seçim	Gerekçe
İkinci iterasyon	pg_qualstats + HypoPG + AST/kural motoru	Index ve SQL rewrite önerileri

4. Genel Sistem Mimarisi



4.1 Kaynak instance - Port 5432

- Uygulama test veritabanı ve gerçek test sorgu yükü burada çalışır.
- pg_stat_statements cluster genelinde normalize sorgu metriklerini toplar.
- Dedicated powa veritabanı, PoWA remote snapshot SQL fonksiyonlarını ve stats extension nesnelerini barındırır.
- Repository geçmişi burada tutulmaz.
- FastAPI doğrudan bu instance'a raporlama sorgusu göndermez.

4.2 Repository instance - Port 5433

- PoWA Repository ve powa-archivist tablolarını barındırır.
- Collector tarafından yazılan snapshot ve aggregate verilerini saklar.
- Kendi advisor şemamız ve hesaplanmış skorlarımız burada tutulur.
- FastAPI için tek okuma noktasıdır.
- 90 günlük retention başlangıç hedefidir; gerçek hacim görüldükten sonra ayarlanır.

4.3 PoWA Collector

- Repository'de tanımlanan remote sunucuları okur.
- Kaynak instance'a bağlanıp snapshot fonksiyonlarını çalıştırır.

- Sonuçları repository instance'a yazar.
- Ayrı işletim sistemi kullanıcısı ve systemd servisi olarak çalışır.
- İlk iterasyonda kendi collector servisimiz yazılmaz.

5. Bileşenler ve Sorumluluklar

Bileşen	Görev	Durum	Yer/bağımlılık
PostgreSQL 5432	Kaynak iş yükü	Zorunlu	pg_stat_statements, powa-archivist paket dosyaları
PostgreSQL 5433	PoWA Repository	Zorunlu	powa, btree_gist, stats extension şemaları
pg_stat_statements	Normalize sorgu metrikleri	Zorunlu	Kaynak instance
PoWA Collector	Periyodik snapshot	Zorunlu	Repository ve kaynak bağlantısı
FastAPI	API ve analiz katmanı	Zorunlu	Repository read-only bağlantısı
Web UI	Dashboard ve kullanıcı etkileşimi	Zorunlu	FastAPI
PoWA Web	Hazır PoWA arayüzü	Kullanılmayacak	-
pg_qualstats	WHERE/JOIN predicate geçmişi	İkinci iterasyon	Kaynak ve repository
HypoPG	Sanal index testi	İkinci iterasyon	Hedef test veritabanı
PEV2	EXPLAIN görselleştirme	İkinci iterasyon	JSON execution plan

6. Kurulum Öncesi Gereksinimler

6.1 Donanım ve işletim sistemi

Kaynak	Pilot önerisi	Not
CPU	En az 4 vCPU	Kaynak ve repository aynı fiziksel sunucuda olduğu için 8 vCPU tercih edilir.
RAM	En az 16 GB	İki PostgreSQL instance ve uygulama katmanı birlikte çalışacaktır.
Disk	En az 150 GB boş alan	Retention ve sorgu çeşitliliğine göre büyütülmelidir.
İşletim sistemi	Desteklenen Linux dağıtımı	Debian/Ubuntu veya RHEL/Rocky örnekleri verilir.
Saat senkronizasyonu	NTP/chrony aktif	Dönem karşılaştırmaları için zorunlu.

6.2 Ortam keşif komutları

```
# İşletim sistemi
cat /etc/os-release

# PostgreSQL istemci sürümü
psql --version

# Mevcut cluster/servisler
systemctl list-units --type=service | grep -i postgres

# PostgreSQL içinden
SELECT version();
SHOW server_version;
SHOW config_file;
SHOW data_directory;
SHOW shared_preload_libraries;
SHOW port;
```

6.3 Kurulum parametreleri

Parametre	Örnek	Açıklama
<PG_MAJOR>	16 veya 17	Kurulu PostgreSQL ana sürümü
<SOURCE_PORT>	5432	Uygulama test instance portu
<REPO_PORT>	5433	PoWA Repository instance portu
<SOURCE_POWA_DB>	powa	Kaynak instance üzerindeki dedicated PoWA DB
<REPO_DB>	powa_repository	Repository veritabanı
<COLLECTOR_USER>	powa_collector	Collector OS/service kullanıcısı
<API_USER>	advisor_api	FastAPI repository read-only rolü
<RETENTION>	90 gün	Başlangıç geçmiş tutma süresi

6.4 Resmî indirme adresleri ve sürüm sabitleme

Kurulum paketleri yalnız resmî PostgreSQL, PoWA Team, GitHub Release ve PyPI kaynaklarından alınmalıdır. Kurulum gününde en güncel kararlı sürüm tekrar doğrulanmalı ve kullanılan tüm paket sürümleri kurulum tutanağına yazılmalıdır. Bu belgenin hazırlandığı 22 Temmuz 2026 tarihinde PoWA Archivist için 5.2.0 ve PoWA Collector için 1.3.2 güncel kararlı sürümlerdir; gelecekte doğrudan “latest” yerine test edilmiş ve sabitlenmiş sürüm kullanılmalıdır.

Bileşen	Resmî adres
PostgreSQL genel indirme	https://www.postgresql.org/download/
Ubuntu paketleri / PGDG	https://www.postgresql.org/download/linux/ubuntu/

Bileşen	Resmî adres
Debian paketleri / PGDG	https://www.postgresql.org/download/linux/debian/
RHEL, Rocky ve AlmaLinux paketleri	https://www.postgresql.org/download/linux/redhat/
PoWA ana dokümantasyonu	https://powa.readthedocs.io/en/latest/
PoWA remote mode kurulumu	https://powa.readthedocs.io/en/latest/remote_setup.html
PoWA Archivist kurulum rehberi	https://powa.readthedocs.io/en/latest/components/powa-archivist/installation.html
PoWA Archivist release arşivi	https://github.com/powa-team/powa-archivist/releases
PoWA Collector release arşivi	https://github.com/powa-team/powa-collector/releases
PoWA Collector PyPI	https://pypi.org/project/powa-collector/
Collector 1.3.2 wheel	https://files.pythonhosted.org/packages/2c/1f/6632aba3a453499f131e599b2eabac3588956c44f1d51a9229763a1d32dd/powa_collector-1.3.2-py3-none-any.whl
Collector 1.3.2 kaynak paketi	https://files.pythonhosted.org/packages/a8/36/2b9eec5dd2b745ab1a9c99e2348f26625049dce2c1aba996f9e02986a332/powa_collector-1.3.2.tar.gz
pg_stat_statements dokümantasyonu	https://www.postgresql.org/docs/current/pgstatstatements.html

Kurulum için “main/master” branch kullanılmamalıdır. PoWA Archivist için bir release etiketi (örneğin REL_5_2_0), Collector için belirli bir PyPI sürümü (örneğin 1.3.2) sabitlenmelidir. İndirilen dosyalarda SHA-256 doğrulaması yapılmalıdır.

6.5 Tercih edilen kurulum rotası

Bu dokümanda önerilen ana rota paket yöneticisiyle kurulumdur. Paket yöneticisi; dosyaları doğru PostgreSQL dizinine yerleştirir, bağımlılıkları çözer ve güncelleme/geri alma işlemlerini kolaylaştırır. Manuel kaynak kod kurulumu yalnız PGDG paketi bulunmadığında veya kurumun offline paket süreci bunu gerektirdiğinde kullanılmalıdır.

1. Mevcut uygulama PostgreSQL instance'ı 5432 portunda korunur.
2. Aynı PostgreSQL ana sürümünde ikinci cluster 5433 portunda oluşturulur.
3. Kaynak instance'a yalnız `pg_stat_statements` ve PoWA remote fonksiyonlarını sağlayan paket dosyaları eklenir.
4. PoWA Repository 5433 üzerinde kurulur.
5. PoWA Collector ayrı bir Python virtual environment ve systemd servisi olarak kurulur.
6. PoWA Web kurulmaz; dashboard daha sonra kendi uygulamamızla geliştirilir.
7. PEV2, `pg_qualstats` ve `HypoPG` ilk iterasyonda kurulmaz; ikinci iterasyonda değerlendirilir.

7. Baştan Sona Paketli Kurulum Runbook'u

Bu bölümdeki komutlar örnek bir Linux test sunucusunda, kaynak PostgreSQL'in 5432 ve repository PostgreSQL'in 5433 portunda çalışacağı varsayımıyla verilmiştir. <PG_MAJOR> ve parola alanları gerçek değerlerle değiştirilmelidir. Önce test ortamında uygulanmalı, ardından kurulum kayıt altına alınmalıdır.

7.1 Kurulumdan önce yedek ve keşif

Önce mevcut PostgreSQL konfigürasyonu, servis durumu ve extension listesi kaydedilir. Kaynak instance üzerinde değişiklik yapılmadan önce konfigürasyon dosyaları yedeklenir.

```
set -euo pipefail

cat /etc/os-release
psql --version
sudo -u postgres psql -p 5432 -d postgres -c "SELECT version();"
sudo -u postgres psql -p 5432 -d postgres -c "SHOW config_file;"
sudo -u postgres psql -p 5432 -d postgres -c "SHOW data_directory;"
sudo -u postgres psql -p 5432 -d postgres -c "SHOW shared_preload_libraries;"
sudo -u postgres psql -p 5432 -d postgres -c \
    "SELECT name, default_version, installed_version FROM pg_available_extensions WHERE
    → name IN ('pg_stat_statements', 'btree_gist', 'powa');"

# Debian/Ubuntu örneği: mevcut config yedeği
sudo cp -a /etc/postgresql /etc/postgresql.backup-$(date +%F-%H%M%S)
```

7.2 Debian / Ubuntu üzerinde PGDG deposunu hazırlama

PostgreSQL ve PoWA paketleri mevcut dağıtım deposunda istenen sürümde bulunmuyorsa PostgreSQL Global Development Group APT deposu kullanılır. Resmî otomatik yöntem:

```
sudo apt update
sudo apt install -y postgresql-common curl ca-certificates
sudo /usr/share/postgresql-common/pgdg/apt.postgresql.org.sh
sudo apt update
```

Otomatik script kullanımına izin verilmeyen kurumlarda resmî anahtar ve source dosyası elle eklenebilir:

```
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc \
    --fail https://www.postgresql.org/media/keys/ACCC4CF8.asc

. /etc/os-release
CODENAME="${VERSION_CODENAME}"
ARCH="$(dpkg --print-architecture)"

sudo tee /etc/apt/sources.list.d/pgdg.sources >/dev/null <<EOF
Types: deb deb-src
URIs: https://apt.postgresql.org/pub/repos/apt
```

```
Suites: ${CODENAME}-pgdg
Architectures: ${ARCH}
Components: main
Signed-By: /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc
EOF

sudo apt update
```

7.3 Debian / Ubuntu paketlerinin kurulması

Kaynak instance zaten kuruluysa aynı ana sürüm için server, client, contrib ve PoWA paketlerinin mevcut olduğu doğrulanır. İkinci cluster için ayrıca başka bir PostgreSQL sürümü kurulmaz.

```
PG_MAJOR=<PG_MAJOR>

sudo apt install -y \
    postgresql-${PG_MAJOR} \
    postgresql-client-${PG_MAJOR} \
    postgresql-contrib-${PG_MAJOR} \
    postgresql-${PG_MAJOR}-powa \
    python3 python3-venv python3-pip

# Paketlerin görünürlüğü
apt-cache policy postgresql-${PG_MAJOR}-powa
```

7.4 Repository cluster'ını 5433 portunda oluşturma

Debian/Ubuntu'da postgresql-common paketinin pg_createcluster aracı kullanılır.

```
PG_MAJOR=<PG_MAJOR>

# Aynı isimde cluster önceden yoksa oluştur
sudo pg_createcluster ${PG_MAJOR} powa_repo --port 5433 --start

# İki cluster'ı doğrula
pg_lsclusters

# Repository cluster durumunu kontrol et
sudo pg_ctlcluster ${PG_MAJOR} powa_repo status
```

Beklenen yerleşim:

```
Kaynak cluster      : <PG_MAJOR>/main      port 5432
Repository cluster  : <PG_MAJOR>/powa_repo port 5433
```

7.5 Kaynak instance'ta pg_stat_statements etkinleştirme

pg_stat_statements ek paylaşımlı bellek istediği için kaynak instance'ın shared_preload_libraries ayarına eklenir ve bir defa restart edilir. Var olan preload kütüphaneleri silinmemelidir.

```
# Kaynak cluster config dosyasını bul
sudo -u postgres psql -p 5432 -Atc "SHOW config_file;"

# postgresql.conf içine örnek ayarlar
shared_preload_libraries = 'pg_stat_statements'
compute_query_id = on
pg_stat_statements.track = top
pg_stat_statements.max = 10000
pg_stat_statements.save = on
pg_stat_statements.track_planning = off
track_io_timing = on
```

Config sözdizimi kontrol edildikten sonra yalnız kaynak cluster restart edilir:

```
sudo pg_ctlcluster <PG_MAJOR> main restart
sudo pg_ctlcluster <PG_MAJOR> main status

sudo -u postgres psql -p 5432 -d postgres -c "SHOW shared_preload_libraries;"
sudo -u postgres psql -p 5432 -d postgres -c "SHOW compute_query_id;"
```

7.6 Kaynak instance üzerinde dedicated PoWA veritabanı

PoWA remote snapshot fonksiyonları ve stats extension nesneleri kaynak instance üzerinde yalnız bir dedicated veritabanında oluşturulur.

```
sudo -u postgres createdb -p 5432 powa

sudo -u postgres psql -p 5432 -d powa <<'SQL'
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE EXTENSION IF NOT EXISTS powa;
SQL

sudo -u postgres psql -p 5432 -d powa -c \
    "SELECT extname, extversion FROM pg_extension ORDER BY extname;"
```

7.7 Collector PostgreSQL rolünü oluşturma

PostgreSQL 10 ve üstünde collector rolü `pg_read_all_stats` üyeliğiyle çalışabilir. PoWA fonksiyonları için gereken EXECUTE ve repository tabloları için gereken yazma yetkileri kurulu PoWA sürümüne göre doğrulanmalıdır. Aşağıdaki başlangıç yetkileri test ortamı içindir; üretimde daha da daraltılmalıdır.

```
# Güçlü parolayı shell history'ye yazmamak için psql içinde girin.
sudo -u postgres psql -p 5432 -d powa <<'SQL'
CREATE ROLE powa_collector LOGIN;
GRANT pg_read_all_stats TO powa_collector;
GRANT CONNECT ON DATABASE powa TO powa_collector;
GRANT USAGE ON SCHEMA public TO powa_collector;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO powa_collector;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT EXECUTE ON FUNCTIONS TO powa_collector;
SQL
```

```
# Parolayı interaktif ayarlayın
sudo -u postgres psql -p 5432 -d postgres -c "\\password powa_collector"
```

7.8 Repository veritabanı ve PoWA Repository kurulumu

Repository cluster'ın kendi performansı bu iterasyonda izlenmeyeceği için 5433 tarafında `shared_preload_libraries` değiştirilmez ve repository için restart gerekmez. Ancak extension dosyaları ve SQL nesneleri bulunmalıdır.

```
sudo -u postgres createdb -p 5433 powa_repository

sudo -u postgres psql -p 5433 -d powa_repository <<'SQL'
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE EXTENSION IF NOT EXISTS powa;
SQL

sudo -u postgres psql -p 5433 -d powa_repository -c \
  "SELECT extname, extversion FROM pg_extension ORDER BY extname;"
```

Repository tarafında collector rolü oluşturulur ve PoWA tablolarına gerekli yetkiler verilir:

```
sudo -u postgres psql -p 5433 -d powa_repository <<'SQL'
CREATE ROLE powa_collector LOGIN;
GRANT CONNECT ON DATABASE powa_repository TO powa_collector;
GRANT USAGE ON SCHEMA public TO powa_collector;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO powa_collector;
GRANT USAGE, SELECT, UPDATE ON ALL SEQUENCES IN SCHEMA public TO powa_collector;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO powa_collector;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO powa_collector;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT USAGE, SELECT, UPDATE ON SEQUENCES TO powa_collector;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT EXECUTE ON FUNCTIONS TO powa_collector;
SQL

sudo -u postgres psql -p 5433 -d postgres -c "\\password powa_collector"
```

PoWA 5, yetkilendirmeyi kolaylaştıran PoWA rolleri sağlayabilir. Kurulu sürümde `du` ve PoWA güvenlik dokümantasyonu kontrol edilerek genel ALL TABLES yaklaşımı yerine hazır rol kullanımı tercih edilmelidir. Buradaki grant seti kurulumun teknik olarak anlaşılması için açık biçimde gösterilmiştir.

7.9 pg_hba.conf ve localhost erişimi

İki cluster aynı sunucuda bulunduğu için bağlantılar 127.0.0.1 ile sınırlandırılır. Her cluster'ın kendi `pg_hba.conf` dosyası düzenlenir.

```
# Kaynak instance 5432 pg_hba.conf
host      powa                powa_collector    127.0.0.1/32      scram-sha-256

# Repository instance 5433 pg_hba.conf
host      powa_repository    powa_collector    127.0.0.1/32      scram-sha-256
host      powa_repository    advisor_api       127.0.0.1/32      scram-sha-256
```

Değişiklik sonrası restart yerine reload yeterlidir:

```
sudo pg_ctlcluster <PG_MAJOR> main reload
sudo pg_ctlcluster <PG_MAJOR> powa_repo reload
```

7.10 Repository'ye kaynak sunucuyu kaydetme

Parolanın `powa_servers` tablosunda düz metin tutulmaması için `password => NULL` kullanılır ve Collector işletim sistemi kullanıcısının `.pgpass` dosyasından yararlanır. PoWA'nın varsayılan remote snapshot aralığı 300 saniyedir.

```
sudo -u postgres psql -p 5433 -d powa_repository <<'SQL'
SELECT powa_register_server(
    hostname => '127.0.0.1',
    port     => 5432,
    alias    => 'test-source',
    username => 'powa_collector',
    password => NULL,
    dbname   => 'powa',
    frequency => 300,
    extensions => '{}'
);
SQL

sudo -u postgres psql -p 5433 -d powa_repository -c \
"SELECT id, hostname, port, alias, username, dbname, frequency FROM powa_servers
↪ ORDER BY id;"
```

7.11 PoWA Collector'ı virtual environment ile kurma

Belge tarihi itibarıyla sabitlenen sürüm 1.3.2'dir. Kurulum gününde release sayfası kontrol edilip kurum tarafından test edilmiş sürüm kullanılmalıdır.

```
sudo useradd --system \
    --home-dir /var/lib/powa-collector \
    --create-home \
    --shell /usr/sbin/nologin \
    powa-collector

sudo install -d -o powa-collector -g powa-collector /opt/powa-collector
sudo python3 -m venv /opt/powa-collector/venv
sudo /opt/powa-collector/venv/bin/python -m pip install --upgrade pip
sudo /opt/powa-collector/venv/bin/python -m pip install 'powa-collector==1.3.2'

/opt/powa-collector/venv/bin/powa-collector.py --version
```


Collector config dosyası resmî olarak aranan ilk konum olan `/etc/powa-collector.conf` içine yazılır:

```
sudo tee /etc/powa-collector.conf >/dev/null <<'JSON'
{
  "repository": {
    "dsn": "postgresql://powa_collector@127.0.0.1:5433/powa_repository"
  },
  "debug": false
}
JSON

sudo chown root:powa-collector /etc/powa-collector.conf
sudo chmod 0640 /etc/powa-collector.conf
```

Collector OS kullanıcısının `.pgpass` dosyası hem kaynak hem repository bağlantısını kapsar:

```
sudo -u powa-collector tee /var/lib/powa-collector/.pgpass >/dev/null <<'EOF'
127.0.0.1:5432:powa:powa_collector:<SOURCE_PASSWORD>
127.0.0.1:5433:powa_repository:powa_collector:<REPOSITORY_PASSWORD>
EOF
sudo chmod 0600 /var/lib/powa-collector/.pgpass
sudo chown powa-collector:powa-collector /var/lib/powa-collector/.pgpass
```

7.12 Collector systemd servisi

Pip paketi `powa-collector.py` scriptini virtual environment'ın `bin` dizinine kurar. Collector config yolu komut satırından verilmez; servis `/etc/powa-collector.conf` dosyasını otomatik bulur.

```
sudo tee /etc/systemd/system/powa-collector.service >/dev/null <<'UNIT'
[Unit]
Description=PoWA Collector
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=powa-collector
Group=powa-collector
Environment=HOME=/var/lib/powa-collector
ExecStart=/opt/powa-collector/venv/bin/powa-collector.py
Restart=on-failure
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=read-only
ReadOnlyPaths=/etc/powa-collector.conf
ReadWritePaths=/var/lib/powa-collector

[Install]
WantedBy=multi-user.target
UNIT

sudo systemctl daemon-reload
```

```
sudo systemctl enable --now powa-collector
sudo systemctl status --no-pager powa-collector
```

7.13 Collector ve repository doğrulaması

Önce kaynakta istatistik üretmek için test sorguları çalıştırılır. Ardından en az iki snapshot aralığı boyunca veri birikmesi doğrulanır.

```
# Kaynak metrikleri
sudo -u postgres psql -p 5432 -d powa -c \
  "SELECT queryid, calls, total_exec_time, mean_exec_time, query FROM
  → pg_stat_statements ORDER BY total_exec_time DESC LIMIT 20;"

# Collector logları
sudo journalctl -u powa-collector --since '-15 minutes' --no-pager

# Repository sunucu kaydı
sudo -u postgres psql -p 5433 -d powa_repository -c \
  "SELECT * FROM powa_servers ORDER BY id;"

# PoWA repository tabloları
sudo -u postgres psql -p 5433 -d powa_repository -c \
  "SELECT schemaname, tablename FROM pg_tables WHERE tablename LIKE 'powa%' ORDER BY
  → tablename;"
```

Başarılı kurulum göstergeleri:

- Kaynakta `pg_stat_statements` satırları görünür.
- Collector servisi active/running durumundadır.
- Collector logunda authentication, missing extension veya version mismatch hatası yoktur.
- Repository'de tanımlı remote server aktif görünür.
- En az iki snapshot sonrası geçmiş sorgu metrikleri oluşur.
- `pg_stat_statements_reset()` periyodik olarak çağrılmaz.

7.14 RHEL / Rocky / AlmaLinux kurulum farkları

RHEL ailesinde PGDG YUM repository kurulum komutu işletim sistemi sürümü ve mimariye göre resmî sayfadan üretilmelidir: <https://www.postgresql.org/download/linux/redhat/>. Aşağıdaki komutlar repository etkinleştirildikten sonraki genel şablondur.

```
PG_MAJOR=<PG_MAJOR>

# Dağıtımın yerleşik PostgreSQL modülü PGDG ile çakışıyorsa devre dışı bırakılır.
sudo dnf -qy module disable postgresql || true

sudo dnf install -y \
  postgresql${PG_MAJOR} \
  postgresql${PG_MAJOR}-server \
  postgresql${PG_MAJOR}-contrib \
  powa_${PG_MAJOR} \
```

```
python3 python3-pip

# Kaynak instance'ın mevcut olduğu varsayılır.
# Repository için ayrı PGDATA:
sudo install -d -o postgres -g postgres /var/lib/pgsql/${PG_MAJOR}/powa_repo
sudo -u postgres /usr/pgsql-${PG_MAJOR}/bin/initdb \
  -D /var/lib/pgsql/${PG_MAJOR}/powa_repo

# postgresql.conf içinde port=5433 ayarlanır.
# Kurum standardına uygun ayrı systemd unit oluşturulur.
```

RHEL ailesinde servis dosyası ve PGDATA yolu dağıtım/PGDG sürümüne göre değişebilir. Resmî PGDG sayfasındaki üretilmiş script ve kurulu paketlerin systemd unit dosyaları esas alınmalıdır; generic komutlarla mevcut PostgreSQL servisi üzerine yazılmamalıdır.

8. Manuel ve Offline Kurulum

8.1 Offline paket setini internet erişimli makinede hazırlama

İzole sunucuya rastgele dosya taşımak yerine sürümü sabitlenmiş tek bir kurulum dizini hazırlanır. İçinde işletim sistemi paketleri, PoWA kaynak kodu veya binary paketi, Collector wheel dosyaları, checksum ve kurulum scriptleri bulunur.

```
mkdir -p postgresql-advisor-offline/{os-packages,powa,python-wheels,config,checksums}

# Collector ve tüm Python bağımlılıklarını indir
python3 -m pip download \
  --dest postgresql-advisor-offline/python-wheels \
  'powa-collector==1.3.2'

# PoWA Archivist kaynak release'i
POWA_TAG=REL_5_2_0
wget -O postgresql-advisor-offline/powa/powa-archivist-${POWA_TAG}.tar.gz \
  "https://github.com/powa-team/powa-archivist/archive/refs/tags/${POWA_TAG}.tar.gz"

# Bütün dosyaların checksum'u
find postgresql-advisor-offline -type f -print0 | sort -z | \
  xargs -0 sha256sum > postgresql-advisor-offline/checksums/SHA256SUMS
```

Debian/Ubuntu için gerekli DEB bağımlılıklarını hazırlama biçimi kurumun repository mirror çözümüne göre seçilir. Basit test için `apt download`; eksiksiz bağımlılık seti için `aptly`, `reprepro` veya kurumun APT proxy/mirror altyapısı tercih edilmelidir. RHEL ailesinde `dnf download --resolve --alldeps` kullanılabilir.

```
# RHEL/Rocky örneği - internet erişimli hazırlık makinesi
sudo dnf install -y dnf-plugins-core
cd postgresql-advisor-offline/os-packages
sudo dnf download --resolve --alldeps \
  postgresql<PG_MAJOR> \
  postgresql<PG_MAJOR>-server \
  postgresql<PG_MAJOR>-contrib \
```

```
powa_<PG_MAJOR> \
python3 python3-pip
```

8.2 Offline Collector kurulumu

İzole sunucuda internet kullanmadan wheel dizininden kurulum yapılır:

```
sudo python3 -m venv /opt/powa-collector/venv
sudo /opt/powa-collector/venv/bin/python -m pip install \
  --no-index \
  --find-links /opt/offline/python-wheels \
  'powa-collector==1.3.2'

/opt/powa-collector/venv/bin/powa-collector.py --version
```

8.3 PoWA Archivist'i kaynak koddan derleme

PGDG paketi kullanılamıyorsa PoWA Archivist yalnız kurulu PostgreSQL sürümünün server header dosyalarıyla derlenir. `pg_config` doğru cluster sürümünü göstermelidir.

```
# Debian / Ubuntu build bağımlılıkları
sudo apt install -y \
  build-essential \
  postgresql-server-dev-<PG_MAJOR> \
  postgresql-contrib-<PG_MAJOR>

# RHEL / Rocky build bağımlılıkları
sudo dnf groupinstall -y "Development Tools"
sudo dnf install -y \
  postgresql<PG_MAJOR>-devel \
  postgresql<PG_MAJOR>-contrib

# Kaynak paketi aç ve kur
cd /opt/offline/powa
tar -xzf powa-archivist-REL_5_2_0.tar.gz
cd powa-archivist-REL_5_2_0

which pg_config
pg_config --version
make clean
make PG_CONFIG="$(command -v pg_config)"
sudo make PG_CONFIG="$(command -v pg_config)" install
```

Birden fazla PostgreSQL sürümü kuruluysa açık yol kullanılmalıdır:

```
# Debian/Ubuntu PGDG örneği
make PG_CONFIG=/usr/lib/postgresql/<PG_MAJOR>/bin/pg_config
sudo make PG_CONFIG=/usr/lib/postgresql/<PG_MAJOR>/bin/pg_config install

# RHEL PGDG örneği
make PG_CONFIG=/usr/pgsql-<PG_MAJOR>/bin/pg_config
sudo make PG_CONFIG=/usr/pgsql-<PG_MAJOR>/bin/pg_config install
```

Kurulumdan sonra iki instance'ta extension dosyalarının görünürlüğü kontrol edilir:

```

sudo -u postgres psql -p 5432 -d postgres -c \
  "SELECT name, default_version FROM pg_available_extensions WHERE name IN
  ↳ ('powa','pg_stat_statements','btree_gist');"

sudo -u postgres psql -p 5433 -d postgres -c \
  "SELECT name, default_version FROM pg_available_extensions WHERE name IN
  ↳ ('powa','pg_stat_statements','btree_gist');"

```

8.4 Checksum ve kurulum kanıtı

İzole ortama taşınan dosyalar kurulmadan önce doğrulanır. Kurulum sonunda kullanılan sürümler bir manifest dosyasına yazılır.

```

cd /opt/offline
sha256sum -c checksums/SHA256SUMS

{
  date -Is
  cat /etc/os-release
  psql --version
  /opt/powa-collector/venv/bin/powa-collector.py --version
  sudo -u postgres psql -p 5432 -d powa -Atc \
    "SELECT extname || '=' || extversion FROM pg_extension WHERE extname IN
    ↳ ('powa','pg_stat_statements','btree_gist') ORDER BY extname;"
  sudo -u postgres psql -p 5433 -d powa_repository -Atc \
    "SELECT extname || '=' || extversion FROM pg_extension WHERE extname IN
    ↳ ('powa','pg_stat_statements','btree_gist') ORDER BY extname;"
} | sudo tee /var/log/postgresql-advisor-install-manifest.txt

```

8.5 Kurulum sonrası geri alma

1. Collector durdurulur: `systemctl disable --now powa-collector`.
2. Repository cluster 5433 durdurulur; kaynak uygulama instance'ı çalışmaya devam eder.
3. Remote server registration gerekiyorsa PoWA'nın delete/purge fonksiyonuyla kaldırılır.
4. Kaynakta `pg_stat_statements` kaldırılmayacaksa restart gerekmez; yalnız Collector kapatıldığında veri toplama sona erer.
5. Tam kaldırma istenirse kaynakta extension drop ve `shared_preload_libraries` değişikliği bakım penceresinde uygulanır.
6. Kaynak uygulama şemasına dokunulmadığı için uygulama verisi için rollback işlemi gerekmez.

9. Güvenlik ve Yetkilendirme

Alan	Kontrol	Uygulama notu
Kaynak DB	Collector dışındaki uygulama erişimini kapat	FastAPI kaynak DB'ye bağlanmamalı

Alan	Kontrol	Uygulama notu
Repository	API için read-only rol	PoWA tablolarına SELECT, advisor tablolarına kontrollü erişim
Parola	Secret store / .pgpass / environment	Kaynak koda veya Git'e yazılmamalı
Sorgu metni	Hassas bilgi kabul et	Normalize olsa bile tablo/kolon isimleri hassas olabilir
Web	Kurumsal kimlik doğrulama	OIDC/LDAP veya reverse proxy auth
Audit	Kullanıcı eylemlerini kaydet	Not ekleme, durum değiştirme, dışa aktarma
DDL	Otomatik çalıştırma yok	İkinci iterasyonda bile insan onayı

Sorgu metni ekranları rol bazlı olmalıdır. Yetkisiz kullanıcılar queryid ve özet metrikleri görebilir, tam SQL metni maskelenebilir.

10. Veri Modeli ve Repository Okuma Katmanı

10.1 PoWA tablolarını değiştirmeme prensibi

PoWA'nın kendi tablolarına kolon veya trigger eklenmez. Ürün verileri ayrı advisor şemasında tutulur. Böylece PoWA sürüm yükseltmeleri daha güvenli olur.

```
CREATE SCHEMA IF NOT EXISTS advisor;

CREATE TABLE advisor.query_annotations (
    server_id      integer      NOT NULL,
    database_id    oid          NOT NULL,
    query_id       bigint       NOT NULL,
    status         text         NOT NULL DEFAULT 'NEW',
    note           text,
    updated_by     text         NOT NULL,
    updated_at     timestampz   NOT NULL DEFAULT now(),
    PRIMARY KEY (server_id, database_id, query_id)
);

CREATE TABLE advisor.audit_log (
    id             bigserial PRIMARY KEY,
    event_time     timestampz NOT NULL DEFAULT now(),
    actor          text NOT NULL,
    action         text NOT NULL,
    object_type    text NOT NULL,
    object_key     text NOT NULL,
    details        jsonb
);
```

10.2 Analitik view katmanı

- advisor.v_query_summary: dönem bazında sorgu özetleri.

- `advisor.v_query_regression`: mevcut dönem ile önceki eş dönem karşılaştırması.
- `advisor.v_query_impact`: normalize edilmiş 0-100 Impact Score.
- `advisor.v_table_health`: `pg_stat_user_tables` ve ilgili istatistiklerden sağlık sinyalleri.
- `advisor.v_collector_health`: son snapshot, gecikme ve hata durumu.

10.3 Repository okuma prensipleri

- API sorguları zaman aralığı ile sınırlandırılır.
- Sorgu listeleri sayfalı döner.
- Ağır aggregation'lar materialized view veya periyodik score tablosuna taşınır.
- PoWA şema ve kolon isimleri sürüme göre değişebileceği için bir adapter/repository sınıfı kullanılır.
- Backend doğrudan PoWA tablolarını frontend'e açmaz; kararlı API modeli üretir.

11. Backend Tasarımı (FastAPI)

11.1 Önerilen teknoloji seti

Katman	Teknoloji	Not
Runtime	Python 3.12+	Kurum standardına göre uyarlanabilir
API	FastAPI	OpenAPI ve hızlı prototipleme
DB sürücüsü	psycopg 3	Async veya sync pool
Migration	Alembic	Sadece advisor şeması için
Validation	Pydantic	API sözleşmeleri
Auth	OIDC/LDAP proxy veya JWT	Kurum kimlik altyapısı
Test	pytest	Unit ve repository integration testleri
Gözlemleme	structlog + Prometheus endpoint	Uygulama logları ve metrikler

11.2 Proje yapısı

```

postgresql-advisor/
|-- backend/
|   |-- app/
|   |   |-- main.py
|   |   |-- api/
|   |   |-- core/
|   |   |-- db/
|   |   |-- repositories/
|   |   |-- analytics/
|   |   |-- scoring/
|   |   |-- health/
|   |   |-- schemas/
|   |-- migrations/

```

```
|  `-- tests/
|-- frontend/
|-- deployment/
|-- sql/
`-- docs/
```

11.3 Temel endpointler

Endpoint	Amaç
GET /api/v1/health	API, repository ve collector sağlık durumu
GET /api/v1/servers	İzlenen sunucu listesi
GET /api/v1/databases	Veritabanı listesi
GET /api/v1/queries	Sorgu listesi, filtreleme, sıralama ve pagination
GET /api/v1/queries/{query_id}	Sorgu detayı, SQL ve dönem karşılaştırması
GET /api/v1/regressions	Performansı gerileyen sorgular
GET /api/v1/tables	Tablo sağlık göstergeleri
PATCH /api/v1/queries/{query_id}/annotation	Not ve inceleme durumu
GET /api/v1/export/queries.csv	Yetkili kullanıcı için dışa aktarma

11.4 Örnek FastAPI ayarları

```
# .env örneği
DATABASE_URL=postgres://adviser_api@127.0.0.1:5433/powa_repository
DEFAULT_WINDOW_HOURS=24
MAX_QUERY_PAGE_SIZE=200
SQL_TEXT_VISIBILITY=authorized
RETENTION_DAYS=90
LOG_LEVEL=INFO
```

12. Web Arayüzü ve Dashboardlar

12.1 Genel bakış dashboardu

Kart/Grafik	Gösterilecek veri	Kullanıcı değeri
Toplam DB zamanı	Seçilen dönemde SUM(total_exec_time)	Genel yükü anlama
Kritik sorgu sayısı	Impact Score eşiği üstü	Öncelik hacmi
En yüksek etkili 10 sorgu	Impact Score sıralaması	Hızlı aksiyon listesi
Regresyon sayısı	Önceki döneme göre kötüleşenler	Yeni problemleri yakalama

Kart/Grafik	Gösterilecek veri	Kullanıcı değeri
Collector gecikmesi	Son snapshot yaşı	Verinin güncelliği
DB yük trendi	Saatlik toplam süre	Pik değil trend analizi

12.2 Sorgu listesi

- Impact Score ve öncelik etiketi.
- Normalize SQL'in ilk satırları.
- Çağrı sayısı, toplam süre, ortalama süre.
- DB yük yüzdesi.
- Shared blocks read/hit, temp writes ve WAL.
- Regresyon yüzdesi.
- İnceleme durumu ve kullanıcı notu.
- Filtreler: sunucu, veritabanı, zaman aralığı, min çağrı, min süre, öncelik.

12.3 Sorgu detay ekranı

- Tam normalize SQL kod bloğu.
- 1 saat / 24 saat / 7 gün / 30 gün trend grafiği.
- Mevcut dönem ve önceki eş dönem karşılaştırması.
- Motor bulguları: yüksek toplam yük, yüksek fiziksel okuma, temp kullanım, regresyon.
- İlişkili veritabanı ve kullanıcı bilgisi.
- İkinci iterasyon için ayrılmış “Öneriler” sekmesi (başlangıçta pasif).
- Kopyala, not ekle, inceleniyor/tamamlandı/reddedildi durumları.

12.4 Sistem sağlığı ekranı

Gösterge	İlk iterasyon yorumu	Öneri durumu
Seq Scan yoğunluğu	Büyük ve sık taranan tabloları işaretler	Plan/index incelemesi önerilir
Dead tuple	Sürekli büyüyen dead tuple eğilimini gösterir	Vacuum/autovacuum incelenir
Autovacuum gecikmesi	Yazma artarken vacuum yapılmayan tablolar	Ayar/uzun transaction kontrolü
Uzun transaction	Yaşı eşik üstü transactionlar	Uygulama transaction yönetimi
Lock bekleme	Bekleyen ve engelleyen session zinciri	Kök blocker incelenir
Az kullanılan index	Sadece aday sinyal	İlk iterasyonda DROP önerilmez

13. Impact Score ve Regresyon Analizi

13.1 Puanın amacı

Impact Score mutlak bilimsel gerçek değil, sorguları inceleme önceliğine göre sıralayan açıklanabilir bir heuristiktir. Her metrik kendi dönemindeki percentile veya min-max normalizasyonu ile 0-100 aralığına getirilir.

```
Impact Score =
  0.40 * Total Execution Time Score
+ 0.20 * Physical Read Score
+ 0.15 * Call Frequency Score
+ 0.10 * Temp Write Score
+ 0.10 * Regression Score
+ 0.05 * WAL Score
```

Puan	Öncelik	Yorum
85-100	Kritik	İlk inceleme kuyruğuna alınır
70-84	Yüksek	Yakın zamanda incelenir
40-69	Orta	Trend ve bağlam takip edilir
0-39	Düşük	Bilgi amaçlı

13.2 Regresyon hesabı

```
current_mean_ms = current_exec_time_delta / current_calls_delta
previous_mean_ms = previous_exec_time_delta / previous_calls_delta

regression_percent =
  100 * (current_mean_ms - previous_mean_ms) / NULLIF(previous_mean_ms, 0)
```

- Minimum çağrı eşiği uygulanmalıdır; az çağrılan sorgularda yüzdelere yanıltıcıdır.
- Çağrı sayısı değişimi ayrı gösterilmelidir; toplam yük artışı trafik artışından kaynaklanabilir.
- Sıfır veya reset dönemleri karşılaştırmadan çıkarılmalıdır.
- Haftanın aynı günü/saatini karşılaştırmak mevsimselliği azaltır.

14. Sistem Sağlığı Kuralları

İlk iterasyon sağlık kuralları kesin DBA kararı değil, inceleme sinyali üretir. Tek metrik yerine tablo boyutu, değişim hızı ve zaman içindeki trend birlikte yorumlanır.

Kural	Şüpheli kombinasyon	Yanlış pozitif riski	İlk aksiyon
Seq Scan	Büyük tablo + yüksek seq_tup_read + sık tarama	Küçük tabloda normal olabilir	Sorgu ve EXPLAIN incele

Kural	Şüpheli kombinasyon	Yanlış pozitif riski	İlk aksiyon
Dead tuple	Mutlak sayı/oran sürekli yükseliyor	Geçici artış normaldir	Autovacuum ve uzun transaction kontrolü
Autovacuum	Update/delete artarken last__autovacuum eski	Değişmeyen tabloda eski olması normal	Tablo bazı ayarları incele
Lock zinciri	Uzun bekleme veya tek blocker çok kişiyi engelliyor	Milisaniyelik kilitler normal	Kök blocker ve transaction süresi
Uzun transaction	Yaş eşliğini aşıyor ve vacuum/lock etkisi var	Uzun rapor sorgusu olabilir	Uygulama akışını incele
Az kullanılan index	idx_scan düşük ve yazma yüksek	İstatistik reseti sonrası yanltıcı	DROP değil, aday etiketi

15. İşletim, İzleme, Yedekleme ve Retention

15.1 Servisler

Servis	Sağlık kontrolü	Restart politikası
PostgreSQL 5432	pg_isready + test query	Manuel/bakım penceresi
PostgreSQL 5433	pg_isready + repository query	Manuel/bakım penceresi
PoWA Collector	systemd active + son snapshot	on-failure
FastAPI	/health endpoint	on-failure
Web UI/Nginx	HTTP health	on-failure

15.2 Yedekleme

- Kaynak uygulama DB yedekleme politikası mevcut süreçle devam eder.
- Repository için günlük pg_dump veya fiziksel yedek alınır.
- advisor şeması kullanıcı notları ve audit içerdiği için özellikle yedeklenir.
- Repository tamamen kaybedilirse kaynak DB etkilenmez; yalnız tarihsel analiz kaybolur.
- Restore testi periyodik yapılır.

15.3 Retention ve kapasite

- Başlangıç retention: 90 gün.
- İlk 2 hafta repository günlük büyüme hızı ölçülür.
- Günlük büyüme x 90 + %30 güven payı ile disk tahmini güncellenir.
- Sorgu çeşitliliği yüksekse pg_stat_statements.max ve repository boyutu birlikte izlenir.

- Düzenli `pg_stat_statements_reset()` yapılmaz; reset rapor sürekliliğini bozar.

16. Test Planı ve Kabul Kriterleri

16.1 Kurulum testleri

1. İki PostgreSQL instance farklı portlarda çalışıyor.
2. Kaynak instance `pg_stat_statements` verisi üretiyor.
3. Kaynak `powa` veritabanında gerekli extensionlar mevcut.
4. Repository `powa_repository` veritabanında PoWA tabloları mevcut.
5. Remote server kaydı repository’de görünüyor.
6. Collector iki instance’a bağlanabiliyor.
7. En az iki snapshot repository’ye yazılıyor.
8. FastAPI repository rolü ile sorgu okuyabiliyor.
9. FastAPI kaynak instance’a bağlantı bilgisi taşıyor.
10. Web arayüzünde normalize SQL ve metrikler görüntüleniyor.

16.2 Fonksiyonel kabul kriterleri

Kod	Kabul kriteri
QUERY-01	Son 24 saat en yüksek toplam süreli sorgular doğru sıralanır
QUERY-02	Normalize SQL kullanıcıya kod bloğu olarak gösterilir
QUERY-03	1/7/30 günlük dönem filtreleri çalışır
QUERY-04	Impact Score hesaplanır ve açıklanabilir bileşenleri gösterilir
REG-01	Önceki eş döneme göre regresyon hesaplanır
HEALTH-01	Seq Scan, dead tuple, autovacuum ve lock sinyalleri gösterilir
OPS-01	Collector gecikmesi dashboardda görünür
SEC-01	Yetkisiz kullanıcı tam SQL metnini göremez
AUDIT-01	Not/durum değişiklikleri audit loga yazılır
SAFE-01	Sistem hiçbir DDL/SQL değişikliğini otomatik uygulamaz

16.3 Performans testi

- Kaynak instance’da `pg_stat_statements` overhead’i test yüküyle ölçülür.
- Collector snapshot süresi ve kaynak üzerindeki query süresi kaydedilir.
- Repository sorguları kaynak instance’a ek rapor yükü oluşturmamalıdır.
- Sorgu listesi API yanıtı 24 saatlik pencerede hedef olarak 2 saniyenin altında tutulur.
- Dashboard ilk açılışı hedef olarak 3 saniyenin altında tutulur.

17. Hata Giderme ve Geri Alma

Belirti	Kontrol / çözüm
pg_stat_statements boş	shared_preload_libraries, restart, CREATE EXTENSION ve compute_query_id kontrol edilir
Collector bağlanamıyor	DSN, pg_hba.conf, parola ve port kontrol edilir
Snapshot gelmiyor	Remote server registration, extension listesi ve collector logları kontrol edilir
SQL metni görünmüyor	Rol yetkileri ve sorgu görünürlüğü politikası kontrol edilir
Repository büyüyor	Retention, snapshot sıklığı ve sorgu çeşitliliği incelenir
Negatif/bozuk delta	pg_stat_statements reset veya server restart dönemi ayrıştırılır
API yavaş	Materialized view, index ve ön hesaplama katmanı uygulanır
PoWA upgrade sorunu	Advisor şeması bağımsız tutulur, backup/restore yapılır

17.1 Geri alma planı

1. FastAPI ve web servisleri durdurulur.
2. PoWA Collector durdurulur ve disable edilir.
3. Repository instance kapatılabilir; kaynak uygulama DB çalışmaya devam eder.
4. pg_stat_statements kapatılacaksa shared_preload_libraries değişikliği ve restart bakım penceresinde yapılır.
5. Extension DROP işlemi yalnız veri saklama kararı ve yedek sonrası uygulanır.
6. Kaynak uygulama şeması değişmediği için uygulama rollback gerektirmez.

18. İkinci İterasyon Yol Haritası

İkinci iterasyon, ilk iterasyonda bulunan yüksek etkili sorgular için “nasıl düzeltilebilir?” sorusuna cevap vermeyi hedefler.

Bileşen	Görev	Kurulum/bağımlılık
pg_qualstats	WHERE ve JOIN predicate geçmişi	Kaynak instance shared_preload + restart
Index Advisor	CREATE INDEX adayları	pg_qualstats ve şema bilgisi
HypoPG	Sanal index kullanımı ve planner cost testi	Hedef/test veritabanında extension
libpg_query / AST	SELECT *, DISTINCT, cast, subquery ve JOIN kalıpları	Kendi kural motoru
EXPLAIN JSON	Plan node, cardinality ve join analizi	Kontrollü plan toplama
PEV2	Plan görselleştirme	Offline/self-host

Bileşen	Görev	Kurulum/bağımlılık
Fayda-risk skoru	Index boyutu, yazma maliyeti ve güven seviyesi	İstatistik ve şema verisi
Doğrulama	Önce/sonra benchmark	Test ortamı ve insan onayı

İkinci iterasyonda bile önerilen CREATE INDEX, DROP INDEX veya SQL rewrite otomatik çalıştırılmamalıdır. Çıktı; SQL, gerekçe, risk, güven seviyesi ve test sonucu ile kullanıcıya sunulmalıdır.

19. Repo ve Kaynak Linkleri

- [1] PostgreSQL indirme sayfası: <https://www.postgresql.org/download/>
- [2] PostgreSQL Ubuntu / PGDG: <https://www.postgresql.org/download/linux/ubuntu/>
- [3] PostgreSQL Debian / PGDG: <https://www.postgresql.org/download/linux/debian/>
- [4] PostgreSQL RHEL / Rocky / AlmaLinux: <https://www.postgresql.org/download/linux/redhat/>
- [5] PostgreSQL pg_stat_statements dokümantasyonu: <https://www.postgresql.org/docs/current/pg-statstatements.html>
- [6] PoWA ana dokümantasyonu: <https://powa.readthedocs.io/en/latest/>
- [7] PoWA mimarisi: <https://powa.readthedocs.io/en/latest/architecture.html>
- [8] PoWA remote setup: https://powa.readthedocs.io/en/latest/remote_setup.html
- [9] PoWA Archivist kurulum rehberi: <https://powa.readthedocs.io/en/latest/components/powa-archivist/installation.html>
- [10] PoWA Collector kurulum rehberi: <https://powa.readthedocs.io/en/latest/components/powa-collector/index.html>
- [11] PoWA Collector GitHub releases: <https://github.com/powa-team/powa-collector/releases>
- [12] PoWA Collector PyPI: <https://pypi.org/project/powa-collector/>
- [13] PoWA Archivist GitHub releases: <https://github.com/powa-team/powa-archivist/releases>
- [14] pg_qualstats GitHub: https://github.com/powa-team/pg_qualstats
- [15] HypoPG GitHub: <https://github.com/HypoPG/hypopg>
- [16] PEV2 GitHub: <https://github.com/dalibo/pev2>
- [17] libpg_query GitHub: https://github.com/pganalyze/libpg_query
- [18] SQLFluff GitHub: <https://github.com/sqlfluff/sqlfluff>

Sürüm ve paket isimleri değişebileceği için gerçek kurulum öncesinde resmi dokümantasyon ve kurulu PostgreSQL ana sürümü tekrar doğrulanmalıdır.

Ek A. Kurulum Kontrol Listesi

- ☐ OS ve PostgreSQL ana sürümü belirlendi.
- ☐ Disk/RAM/CPU kapasitesi kontrol edildi.
- ☐ Kaynak 5432 ve repository 5433 port planı onaylandı.
- ☐ Resmî indirme adreslerinden sürümü sabitlenmiş paket seti hazırlandı.
- ☐ Paketlerin SHA-256 değerleri doğrulandı ve kurulum manifesti oluşturuldu.
- ☐ Kaynak instance pg_stat_statements yapılandırıldı ve restart edildi.
- ☐ Kaynak powa veritabanı ve extensionlar oluşturuldu.
- ☐ Repository instance ve powa_repository oluşturuldu.
- ☐ Repository extensionlar oluşturuldu.
- ☐ Collector ve API rolleri oluşturuldu.
- ☐ pg_hba.conf kuralları uygulandı.
- ☐ Remote server repository'ye kaydedildi.
- ☐ PoWA Collector systemd servisi çalışıyor.
- ☐ İki snapshot repository'ye yazıldı.
- ☐ Advisor şeması migrationları uygulandı.
- ☐ FastAPI health endpoint başarılı.
- ☐ Dashboard sorgu listesi açılıyor.
- ☐ Retention 90 gün olarak doğrulandı.
- ☐ Backup ve rollback testi yapıldı.
- ☐ Kabul kriterleri imzalandı.

Ek B. Örnek API Sözleşmesi

```
GET /api/v1/queries?window=7d&sort=impact&limit=50
```

```
{
  "items": [
    {
      "serverId": 1,
      "databaseId": 16384,
      "queryId": 184,
      "sql": "SELECT * FROM orders WHERE customer_id = $1 AND status = $2",
      "calls": 1850000,
      "totalExecTimeMs": 136800000,
      "meanExecTimeMs": 74.0,
      "dbLoadPercent": 18.2,
      "sharedBlocksRead": 9820000,
      "tempBlocksWritten": 0,
      "regressionPercent": 42.0,
      "impactScore": 87,
      "priority": "HIGH",
    }
  ]
}
```

```

    "status": "NEW",
    "findings": [
      "Toplam veritabanı süresinin önemli bölümünü tüketiyor",
      "Önceki eş döneme göre performansı geriledi",
      "Fiziksel okuma miktarı yüksek"
    ]
  },
  "page": 1,
  "pageSize": 50,
  "total": 418
}

```

Ek C. Örnek Ekran Alanları

Ekran	Zorunlu alanlar
Genel bakış	DB zamanı, kritik sorgu, regresyon, collector güncelliği, trend
Sorgu listesi	Impact, SQL, calls, total/mean time, load %, reads, temp, regression, status
Sorgu detayı	Tam SQL, grafikler, dönem karşılaştırması, bulgular, not ve durum
Sistem sağlığı	Seq Scan, dead tuple, autovacuum, uzun transaction, lock, index sinyali
Operasyon	Collector durumu, son snapshot, repository boyutu, retention, servis health
Ayarlar	Eşikler, varsayılan pencere, SQL görünürlüğü, kullanıcı/rol politikası

Bu belge ilk iterasyon için kurulum ve geliştirme referansıdır. Gerçek paket komutları, PostgreSQL ana sürümü ve Linux dağıtımı kesinleştirildiğinde “ortama özel kurulum runbook” belgesiyle tamamlanmalıdır.